MTech Rocks: Optimizing the ext4/JBD2 Consistency Mechanism on Linux 6.1.4

Milan Roy (241110042) Sahil Basia (241110061) Shrey Sharma (251110068)
Suyamoon Pathak (241110091)

CS614 Linux Kernel Programming
Indian Institute of Technology Kanpur

April 2026

Abstract

This is the project report of the **MTech Rocks** group for CS614 (Linux Kernel Programming) at IIT Kanpur. The four members of the group worked on the ext4/JBD2 consistency mechanism from four different angles. Suyamoon worked on two coverage gaps in the fast commit feature (Sections 4–7). Sahil worked on the global `j_state_lock` that every fsync waiter takes inside `jbd2_log_wait_commit` (Section 8). Milan measured the baseline cost of each ext4 journaling mode on WAL and concurrent workloads (Section 9). Shrey worked on the journal layout, replacing the strict ring buffer with a free-block bitmap that lets a transaction use any free journal block (Section 10).

Suyamoon's part has four candidate optimizations. Two of them (C1 and C2) produced no measurable improvement and are kept as postmortems. The other two are kernel patches that close two fast-commit fallback paths: `EXT4_FC_REASON_XATTR` (a 108-line patch on `xattr.c` and `fast_commit.c`) and `EXT4_FC_REASON_FALLOC_RANGE` (a 46-line patch on `extents.c`). On a Linux 6.1.4 VM, the xattr patch reduces the number of full JBD2 commits on a 5000-op `setxattr` workload by 64× (5000 → 78) and cuts wall time by 27%. The fallocate patch reduces full commits on a 1000-op `fallocate(COLLAPSE_RANGE)`+`fsync` workload by 62.5× (1001 → 16) and cuts wall time by 23%, with the INSERT_RANGE numbers nearly identical. Bare-metal runs reproduce the same commit-reduction ratios with larger wall-time gains: 48% for xattr and 43–44% for fallocate. The two patches touch disjoint files and compose without conflict. No new regressions appear in xfstests, and all six custom crash-recovery tests pass.

Sahil's part is an out-of-tree kernel module (`jbd2_trace.ko`) that uses `kretprobes` to do two things. With `optimize=0` it measures how much of total `fsync()` time is spent inside `jbd2_log_wait_commit`; the answer is about 78% on the stock kernel at 16 threads. With `optimize=1` it replaces that function with a lockless double-checked-locking + Compare-And-Swap version. At 16 threads on ext4 `data=ordered` on an NVMe partition, lock-contention IOPS goes up by 40%, metadata IOPS by 54%, and database OLTP IOPS by 7%, with no regression on streaming workloads.

Milan's part is a five-run averaged characterization of all four ext4 modes (`ordered`, `journal`, `writeback`, `nojournal`) on a single-thread WAL workload and on a concurrent WAL+sequential workload, with `bpftrace` attached to the JBD2 tracepoints throughout. The main finding is that on a small-block fsync-heavy WAL workload, journaling halves IOPS regardless of mode, and 99.6% of total per-op latency is the `fdatasync` portion. The cost of journaling on this workload is the cost of the commit, not of the write.

Shrey's part is a loadable kernel module that replaces JBD2's ring-buffer journal allocator with a free-block bitmap. Two small JBD2 patches (about 50 lines together) add the callback hooks the module needs. With the new allocator on, a transaction's blocks can land anywhere in the journal area, not just at the head pointer. P99 latency on the WAL workload drops from 0.409 ms to 0.053 ms in `ordered` mode (about 8×) at the same write amplification factor; `writeback` stays the same; and `journal` mode pays a small cost (3.50× → 3.61× WAF) because the on-disk index block is itself journaled.

Contents

1 Introduction

The goal of this project is to optimize the consistency mechanism of ext4 on Linux 6.1.4. The consistency mechanism here refers to JBD2, the journaling layer that ext4 uses to keep the filesystem crash-consistent, and to the fast-commit extension introduced in Linux 5.10 that provides a lower-latency commit path for `fsync`. The project requires that any change we make must preserve the existing crash-recovery guarantees and must be validated by measurement on Linux 6.1.4, with evaluation done on a VM and, where useful, on bare-metal hardware.

The four members of **MTech Rocks** worked on four independent parts of this mechanism. Each part has its own section of this report:

- **Suyamoon** worked on two coverage gaps in the fast-commit feature: the fallback on every `setxattr` (Section 6) and the fallback on every `fallocate(COLLAPSE_RANGE)` or `INSERT_RANGE` (Section 7). Two earlier candidates (Sections 4 and 5) gave no measurable improvement and are kept as postmortems. The two patches that did work are written up end-to-end with measurements on a VM and on bare-metal hardware.
- **Sahil** worked on the global `j_state_lock` that every fsync waiter takes inside `jbd2_log_wait_commit` (Section 8). The deliverable is an out-of-tree kernel module that *measures* the lock overhead with `kretprobes` and *replaces* the function with a lockless double-checked-locking + Compare-And-Swap version, picked by a module-load parameter.
- **Milan** did not change the kernel. He measured the cost of each ext4 journaling mode (`ordered`, `journal`, `writeback`, `nojournal`) on a single-thread WAL workload and on a concurrent WAL+sequential workload, with `bpftrace` attached to JBD2 throughout (Section 9). The numbers say how much of the per-op cost on this workload is the journal.
- **Shrey** worked on the journal layout itself (Section 10). His loadable module replaces JBD2's strict ring-buffer block allocator with a free-block bitmap, so a transaction's blocks can land anywhere in the journal area. Two small JBD2 patches add the callback hooks the module needs.

The four parts are independent. Suyamoon's two patches touch `fs/ext4/` files only; Sahil's module hooks JBD2 via `kretprobes` without changing the in-tree code; Milan's harness only mounts and measures; Shrey's two patches add JBD2 hooks but the allocator itself lives in his out-of-tree module. Each section starts with the deliverables and how to reproduce them.

1.1 What we built

The deliverables in this submission are:

- **Suyamoon:** a 108-line kernel patch `fc-inline-xattr.patch` against `fs/ext4/xattr.c` and `fs/ext4/fast_commit.c` (Candidate 3); a 46-line kernel patch `fc-fallocate-range.patch` against `fs/ext4/extents.c` (Candidate 4); two primary benchmark scripts (`bench_xattr.sh`, `bench_fallocate_range.sh`); a characterization microbench harness used to pick C4 (`bench_xattr_block.sh`, `bench_concurrent_fsync.sh`); six custom crash-recovery tests (`c3_crash_test_*`, `c4_crash_test_*`); and bare-metal evaluation scripts and instructions. Two earlier patches (C1 and C2) are kept as postmortems.
- **Sahil:** an out-of-tree kernel module `jbd2_trace.ko` (`sahil/module/jbd2_trace.c`) with a Makefile, an automated benchmark matrix (`run_evals.sh` covering ordered/writeback $\times$ opt 0/1 $\times$ {1,4,8,16} threads $\times$ 3 runs over fio, sysbench, fs_mark, and dbench), a `validation.sh` fsck integrity check, a `dmesg_sampler.sh` lock-overhead extractor, and Python aggregation/plotting scripts.

- **Milan:** two reproducible `bpftrace`+fio benchmark suites (`milan/benchmarks/wal_workload_no_fc/` and `milan/benchmarks/combined_workload_no_fc/`), each with N-run averaging scripts and per-block-size averaged result tables.
- **Shrey:** two small JBD2 patches (`shrey/patches/First.patch`, `Second.patch`) that add allocator and free-txn callback hooks; an out-of-tree kernel module `ext4_tracker.ko` (sources in `shrey/module/ext4_tracker.tar.gz`) that registers a new `ext4_tracker` filesystem and, with mount option `-o fjm`, replaces the JBD2 ring-buffer journal allocator with a free-block bitmap; and a five-run benchmark harness comparing `_std` and `_fjm` across three ext4 modes (`shrey/benchmarks/`).

2 Background

2.1 ext4 and JBD2

ext4 is the default Linux journaling filesystem. The original design and roadmap are described by Mathur and colleagues in their 2007 Linux Symposium paper [1], which also introduces JBD2 as the journaling block device that ext4 layers on top of. When you write to a file and then call `fsync()`, ext4 uses JBD2 to make sure the change is on disk before `fsync` returns. A JBD2 "commit" is the operation that flushes a batch of metadata changes through the journal.

A full JBD2 commit is expensive. In our VM it takes around 6 ms. On a real SSD it takes less, but it is still a whole set of writes plus a `REQ_PREFLUSH` to the device. For a workload that does many small `fsync()`s in a loop (like SELinux relabeling or a mail server writing mailboxes), these full commits dominate the runtime.

2.2 Fast Commit

Ts'o and colleagues [5] added the "fast commit" feature to ext4 and JBD2 in Linux 5.10 to address this problem. Fast commit keeps a small reserved region of the journal (the "FC area", default 1.5% of the journal). Instead of writing a full metadata transaction through the regular JBD2 pipeline, fast commit writes a smaller record describing just the operation (for example, "this inode number now has this raw inode content"). It uses eight tag types: `ADD_RANGE`, `DEL_RANGE`, `CREAT`, `LINK`, `UNLINK`, `INODE`, `HEAD`, and `TAIL`.

The ATC 2024 paper by Shirwadkar, Kadekodi, and Ts'o [5] reports that fast commit handles about 98% of fsyncs in typical mail-server and NFS workloads, giving roughly 2.8× fsync speedup. On the 2% it cannot handle, the code calls `ext4_fc_mark_ineligible` and falls back to a full JBD2 commit. The paper lists 10 reasons the fallback triggers, including xattr changes, cross renames, resize, journal flag changes, and encrypted filenames.

2.3 The xattr fallback

This project focuses on one specific fallback reason: `EXT4_FC_REASON_XATTR`. Every call to `setxattr(2)` or `removexattr(2)` on an ext4 filesystem with fast commit enabled forces a full JBD2 commit. The ATC 2024 paper mentions this as a known gap ("only 8 tag types vs XFS's 40"). There is no existing in-tree kernel patch for it.

This matters because xattrs are used everywhere. SELinux labels every file via `security.selinux`. POSIX ACLs use `system.posix_acl_access`. Package managers use `user.*` attributes for checksums and metadata. `systemd-tmpfiles` writes xattrs during boot. When any of these happen on a fast-commit filesystem, the per-operation cost is paid even though the fast commit area is sitting unused.

3 Four Candidates

We did not start with the xattr gap. We got there after two failed attempts. The fourth followed only after the third worked. Each attempt taught us something about how to pick an optimization target. This section describes all four in order.

3.1 How we picked the first three candidates

We code-reviewed the entire JBD2 and ext4 source tree for comments of the form `TODO:`, `FIXME:`, and `WARN_ON(1)`. We also read the ATC 2024 paper's future-work section. We wanted candidates that (a) were backed by developer comments in the code itself, (b) were not already fixed in newer upstream kernels, and (c) could be implemented in a bounded time budget without touching on-disk format.

This gave us three promising candidates: (1) a developer TODO in `fs/jbd2/commit.c:454` about fast commit blocking too early, (2) a developer TODO in `fs/ext4/mballoc.c:2564` about synchronous bitmap prefetch completion, and (3) the xattr fast commit gap.

We attempted them in that order. C1 and C2 produced null measurable results (Sections 4 and 5). C3 worked.

3.2 How the fourth candidate emerged

After C3 succeeded, we examined the remaining `ext4_fc_mark_ineligible` call sites for another path with the same property: a 100% hit rate on a well-defined operation class, a small expected patch size, and a microbench that could distinguish the patched and unpatched behavior. Three candidates were measured against this criterion on the C3 kernel before any kernel code was written (Section 7.1). The winner was the `FALLOC_RANGE` fallback, which became Candidate 4.

4 Candidate 1: Fast Commit Barrier Deferral

4.1 The TODO

In `fs/jbd2/commit.c` around line 454, the JBD2 developers themselves wrote this comment:

```
/*
 * TODO: by blocking fast commits here, we are increasing
 * fsync() latency slightly. Strictly speaking, we don't need
 * to block fast commits until the transaction enters T_FLUSH
 * state. So an optimization is possible where we block new fast
 * commits here and wait for existing ones to complete
 * just before we enter T_FLUSH. That way, the existing fast
 * commits and this full commit can proceed parallely.
 */
```

The current code blocks any in-flight fast commit at `T_LOCKED` (the very first state of a full commit). The developer comment says we could safely let them continue until `T_FLUSH`, which runs several states later. If our full commit and a concurrent fast commit overlap in those states, we save time.

4.2 What we implemented

A three-line change in `fs/jbd2/commit.c`: move the drain loop that waits for in-flight fast commits from before `T_LOCKED` to just before `T_FLUSH`. We also had to move `journal->j_fc_off = 0` because that variable is used by fast commit as an allocation offset into the FC area, and resetting it while a fast commit is still running would corrupt the log. This hidden dependency is why the naive move without tracking `j_fc_off` breaks correctness.

The patch and design are in `suyamoon/jbd2-fc-barrier-defer.patch` and `suyamoon/CANDIDATE1_postmor`

4.3 Why it did not work

The patch is correct. We confirmed this on the VM: it builds, boots, mounts, and runs. No dmesg errors.

The problem was the measurement. On our fio workload (`randwrite -fsync=1 -numjobs=4`) we saw:

- Stock: 6590 μs average commit time.
- Patched: 6661 μs average commit time.

The delta is 1% in the wrong direction, well inside measurement noise. The `/proc/fs/jbd2/loopX-8/info` file showed `0ms transaction was being locked` on the stock kernel. That means the old barrier almost never fired in the first place on this workload, because `T_LOCKED` already takes less than a millisecond. If the code path we optimized is hit less than 1% of the time, no measurement will show a speedup.

4.4 The fast_commit feature-bit oversight

While preparing the patch for evaluation on bare-metal, we noticed that the `mkfs.ext4` command used to format the test filesystem did not include the `-O fast_commit` option. Without that flag the feature bit is not set on the superblock, so `JBD2_FAST_COMMIT_ONGOING` never becomes true, and the drain loop we had moved was never actually executed by the kernel. An earlier "57% improvement" result was a measurement artifact caused by running the stock benchmark for 5 MB while running the patched benchmark for 60 seconds; the two runs were not comparable.

After re-creating the filesystem with `-O fast_commit` enabled and re-running the benchmark, the numbers reduced to the ones reported above: no measurable improvement. Candidate 1 was then retired.

4.5 Lesson

Before investing in a patch, confirm that (a) the feature the patch depends on is enabled in your benchmark, and (b) the slow path you are optimizing is actually hit often in your workload. Candidate 1 failed both tests.

5 Candidate 2: Async Bitmap Prefetch Completion

5.1 The TODO

In `fs/ext4/mballoc.c:2564`, the ext4 block allocator developers left this comment:

```
/*
 * TODO: We should actually kick off the buddy bitmap setup in a work
 * queue when the buffer I/O is completed, so that we don't block
 * waiting for the block allocation bitmap read to finish when
 * ext4_mb_prefetch_fini is called from ext4_mb_regular_allocator().
 */
```

`ext4_mb_prefetch_fini()` runs at the end of every call to `ext4_mb_regular_allocator`. It walks a list of groups for which we had prefetched bitmaps earlier and calls `ext4_mb_init_group` on each. That function blocks on bitmap I/O if the prefetched read is not yet complete.

5.2 What we implemented

We added a per-superblock workqueue and slab cache. In `prefetch_fini`, we check if the bitmap buffer is uptodate: if yes, do the synchronous init inline; if no, allocate a work item and queue it. The caller then returns without waiting.

Careful correctness work: `ext4_mb_init_group` already handles concurrent callers via the `EXT4_MB_GRP_NEED_INIT` flag and the buddy page lock, so no new locking was needed. But our first version had two real bugs caught by code review:

1. The patch broke the `static noinline_for_stack` attribute on `ext4_mb_init_group` because we inserted our new struct definition between `static noinline_for_stack` and the function signature. The attribute then applied to our struct instead, silently removing the stack-isolation the original author wanted.

2. On `ext4_mb_init` OOM, the error path leaked `s_buddy_cache` because the existing `out_free_locality_groups` label did not know how to undo the backend init.

Both were fixed. The patch is at `suyamoon/mballoc-async-prefetch.patch`.

5.3 Why it did not work (on the VM)

The patch is correct and the TODO it implements is real. But on our VM we could not measure any improvement. Two reasons:

- Our fallocate-based microbenchmark does not trigger cold group initialization often enough. After the first few allocations, the bitmaps are in memory and `prefetch_fini` takes the fast path, which is unchanged by our patch.

- Our fio throughput benchmark showed 70% coefficient of variation across 5 repeats on the same kernel, because the VM's loop device spills to the host page cache non-deterministically. At 70% CV, no patch can be measured unless it is a 2× speedup or more.

We wrote bare-metal evaluation scripts but did not run them on a patch whose signal we could not even confirm on the VM. The postmortem is at `suyamoon/CANDIDATE2_postmortem.md`.

5.4 Lesson

Do not optimize rare paths. Check the hit rate of the slow path in your benchmark before investing. For Candidate 2, the slow path (async bitmap init needed) was hit by less than 1% of operations in our workload.

Also: if your measurement setup has 70% CV, improve the setup before running experiments. You cannot see a small effect through large noise.

6 Candidate 3: Fast Commit Support for Inline xattrs

After two failed attempts, we changed our selection criteria. Now we looked for a slow path that is hit on 100% of a specific kind of operation, not just an occasional one. We wanted a path where we could say "every time operation X happens, we pay this cost, and the patch removes it."

The xattr fast-commit fallback fits exactly. Every single `setxattr`/`removexattr` on a fast-commit-enabled filesystem forces a full JBD2 commit.

6.1 Code analysis

We traced the code path in `fs/ext4/xattr.c`. The key function is `ext4_xattr_set_handle`. At its end (line 2422), regardless of success or failure, regardless of whether the operation was inline or block, it unconditionally calls:

```
ext4_fc_mark_ineligible(inode->i_sb, EXT4_FC_REASON_XATTR, handle);
```

That single line kills fast commit for the entire transaction.

We found a second `mark_ineligible` at line 2500 inside the `ext4_xattr_set` wrapper function, which runs after `ext4_journal_stop`. This was more subtle and almost made our patch a third null result.

6.2 Why inline xattrs are the interesting case

xattrs in ext4 live in three places:

1. **Inline**: in the inode body itself, between `EXT4_GOOD_OLD_INODE_SIZE + i_extra_isize` and the end of the inode. On a 256-byte inode this gives about 80 bytes for xattrs. Most SELinux labels, POSIX ACLs, and short `user.*` xattrs fit here.

2. **Block**: in a separate 4 KB block pointed to by `i_file_acl`. Used when xattrs do not fit inline.

3. **EA inode**: each xattr gets its own small inode. Used when the value is very large and the `ea_inode` feature is enabled.

Our patch handles case (1). For cases (2) and (3) we keep the full commit fallback. This covers nearly all real-world xattr operations because SELinux and ACLs are always short and fit inline.

6.3 Design

The design has two parts, both small.

Part 1. Expand what fast commit logs per inode. Today, `ext4_fc_write_inode` at `fs/ext4/fast_commit.c:863` logs the inode's raw bytes up to `EXT4_GOOD_OLD_INODE_SIZE + i_extra_isize`. That stops just before the inline xattr region. We add a two-line conditional that expands the log to `EXT4_INODE_SIZE` when the inode has inline xattrs. We detect inline xattrs using the pre-existing `EXT4_STATE_XATTR` flag, which ext4 already sets and clears correctly in its xattr code.

On the replay side, no changes are needed. The existing replay code at `ext4_fc_replay_inode` copies `fc_len` bytes from the log into the inode, and the validator already accepts records up to `s_inode_size`. When our patched kernel logs a larger inode, the same replay code restores it correctly. This is the main reason the patch came out so small.

Part 2. Conditionally skip the ineligibility call. We add a local `bool touched_block = false` at the top of `ext4_xattr_set_handle`. Every time the function calls `ext4_xattr_block_set` or takes the EA-inode path, we set `touched_block = true`. At the tail, we replace the unconditional `mark_ineligible` call with:

```
if (error || touched_block || !ext4_has_feature_xattr(inode->i_sb))
    ext4_fc_mark_ineligible(inode->i_sb,
                            EXT4_FC_REASON_XATTR, handle);
```

The three conditions are:

- `error`: the set operation failed; fall back to be safe.
- `touched_block`: we modified a separate xattr block or an EA inode; fast commit does not log these, so we must full-commit.
- `!ext4_has_feature_xattr(sb)`: this is the first xattr on a freshly-mkfs'd filesystem. The superblock xattr feature bit has just been set in memory but not yet persisted. If we fast-commit now and the machine crashes, the replayer would not see the feature bit. So the first xattr is always a full commit.

The `xattr.c:2500` fix. This is the change we almost missed. The wrapper `ext4_xattr_set` had its own unconditional `mark_ineligible(..., NULL)` call that ran after `ext4_journal_stop`. With `handle==NULL`, the call marks the current running JBD2 transaction (or the next) as ineligible, which would suppress fast commit on any following `fsync` that came through the same transaction. If we had left this in, our patch would have done nothing for end-users even though it was correct inside `set_handle`. We deleted the call after confirming its job was already handled by our new conditional at line 2422 on all paths where the inode was actually modified.

6.4 Implementation

The patch is 108 lines in a unified diff format. It touches two files:

- `fs/ext4/fast_commit.c`: `+9/-0` (the new conditional with its comment).
- `fs/ext4/xattr.c`: `+27/-4` (the `touched_block` declaration, five assignments to it on each block-touching path, the new conditional at line 2422, the removal of the redundant `mark_ineligible` call at line 2500, and explanatory comments at both sites).

The full patch is at `suyamoon/fc-inline-xattr.patch`.

6.5 Correctness verification

We did four layers of correctness checking.

Code review. The patch went through a spec-compliance review and a code-quality review. The reviewer caught a subtle ineligibility-marker that we missed in our first draft: the `ext4_xattr_set` wrapper had a second `mark_ineligible` call after `ext4_journal_stop` (Section 6.3, "the `xattr.c:2500` fix"). Without removing that call, our patch would have done nothing for end-users even though the change inside `ext4_xattr_set_handle` was correct. Catching this before merge avoided a third null result.

Compilation. The patched files compile with zero warnings at kernel warning level 2.

Custom crash recovery tests. We wrote three small shell scripts that specifically exercise our changed paths:

- `c3_crash_test_a.sh`: set 100 inline xattrs on a fast-commit filesystem, simulate a crash via lazy unmount, remount, check that all 100 are there. **PASS.**
- `c3_crash_test_b.sh`: set 100, remove the 50 even-numbered ones, simulate a crash, remount, check that exactly the 50 odd ones remain. **PASS.**

- `c3_crash_test_c.sh`: write a 500-byte xattr which forces the block path (our patch should *not* take the fast path), simulate a crash, remount, check that all 668 bytes of the value survive. **PASS.** This test confirms the fallback branch works: large xattrs still use full commit and still survive a crash.

xfstests subset. We ran `generic/{062, 118, 300, 337, 454, 455, 473, 482}` and `ext4/032` on both the stock and patched kernels. Six of the nine tests apply to our setup. Five pass on both kernels. One (`generic/473`, a `SEEK_DATA` hole-finding test) fails on *both* stock and patched with the same output diff (`1: [128..135]: data` vs expected `1: [128..255]: data`). We therefore classify it as a pre-existing ext4 bug in 6.1.4 with `fast_commit`, unrelated to our patch. The other three tests require features we do not have enabled (reflink, `LOGWRITES_DEV`). Full logs are in `suyamoon/xfstests_c3/`.

We deliberately skipped `generic/388` because it is an XFS-specific log-recovery test and its `fsstress` phase hangs on ext4+fast_commit regardless of our patch.

6.6 Performance evaluation: VM

Test setup:

- VirtualBox VM, 11th-gen Intel Core i7-11800H, 4 cores, 5.7 GiB RAM.
- 256 MB loop device backed by a host ext4 filesystem.
- Filesystem: `ext4 -O fast_commit`.
- Workload: a small C helper does 5000 `fsetxattr(fd, "user.test", ...)` + `fsync(fd)` iterations on the same file. Per-op `fsync` is essential because without it all 5000 ops collapse into one JBD2 transaction and the benchmark measures nothing.

Results:

Metric	Stock	Patched	Change
Wall time (5000 ops)	31,102 ms	22,734 ms	−26.9%
Per-op latency	6,220 μs	4,546 μs	−26.9%
JBD2 full commits	5,000	**78**	**−98.4%**
commits_per_op	1.00	0.016	–

The 64× reduction in full commits (from 5000 to 78) is the clean architectural signal. Our patch engages the fast commit path on 98.4% of operations, exactly as designed. The remaining 1.6% are the expected cases: one full commit for the warmup xattr (feature-bit persistence, by design), and a handful from JBD2's natural 5-second timer and fast-commit-area wraparounds.

The 27% wall-time reduction is smaller than we initially hoped. Analysis: on the VM, each `fsync` takes about 5 ms end-to-end, of which about 1.7 ms is the JBD2 full commit we eliminate. The remaining 3.3 ms is VFS, syscall entry/exit, and the host-backed loop device's write-behind path. Our patch cannot touch any of that. On real hardware, the ratio shifts.

6.7 Performance evaluation: bare-metal

We re-ran the same benchmark on bare-metal hardware: 11th Gen Intel Core i3-1115G4, 4 cores, 7.5 GiB RAM, real SATA SSD. Three repeats of each configuration.

Metric	Stock	Patched	Change
Wall time (mean)	57,627 ms	30,089 ms	−47.8%
Stdev	3,225 ms	2,088 ms	(∼6% CV)
Per-op latency	11,525 μs	6,017 μs	−47.8%
JBD2 full commits	5,000	**78**	**−98.4%**

Two observations.

First, the transaction count reduction is identical to the VM result: 64×, 5000 → 78. This means the patch engages fast commit on the same set of operations regardless of the hardware. This is the architectural claim of the patch, and it reproduces across machines with different CPUs, memory, and storage.

Second, the wall-time speedup is nearly doubled on the bare-metal machine: 48% versus 27%. This matches the analysis in the VM section. On a real SSD the VFS and syscall overhead form a smaller share of per-op cost, so eliminating the JBD2 commit portion gives a larger relative gain.

The 3-repeat coefficient of variation on bare-metal is about 6%, which is a usable measurement setup. The result is not a one-off artifact.

6.8 Non-xattr regression check

To confirm our patch does not regress unrelated workloads, we ran `fio -rw=randwrite -bs=4k -fsync=1 -numjobs=4` for 30 s on the patched kernel and compared against the stock baseline. Stock: 530 IOPS. Patched: 528 IOPS. Delta 0.4%, well inside noise. No regression.

7 Candidate 4: Fast Commit Support for fallocate Range Operations

After Candidate 3 succeeded we asked a simple question: is there another slow path in ext4 with the same shape? That is, an operation that (a) is used regularly on a fast-commit filesystem, (b) hits the `ext4_fc_mark_ineligible` fallback on 100% of calls, and (c) has a simple enough code path that the fix is a small patch, not a rewrite.

We found one: the `FALLOC_FL_COLLAPSE_RANGE` and `FALLOC_FL_INSERT_RANGE` modes of `fallocate(2)`. These are the modes that, respectively, remove a middle slice of a file and shift following data left, or insert a zero-filled slice at a given offset and shift following data right. They are used by log rotators, database log compactors, and VM disk tools. Both modes call `ext4_fc_mark_ineligible(EXT4_FC_REASON_FALLOC_RANGE)` unconditionally and force a full JBD2 commit on every invocation.

7.1 Measure first, then decide

The cost of our first two failures was that we implemented patches before confirming the slow path was actually hit hard enough to measure. For Candidate 4 we flipped the order. We built three microbenchmarks and ran them on the C3 kernel to compare three possible targets:

- fallocate COLLAPSE / INSERT range (the target described above);
- "block xattr": large (4 KB) `setxattr` values that spill out of the inode and into a separate xattr block, which our Candidate 3 patch does not cover;
- concurrent `fsync` from many threads, which is the CJFS-style compound flush target.

The rule we used: only implement a patch if the microbench shows at least `tx/op` $\geq$ `0.5` on the candidate path. That is, at least one JBD2 full commit per op. Anything below means the slow path is already being coalesced and our patch would not change much.

Results (3 runs each, N = 1000 ops or 100 per thread):

Workload	ms/op	tx/op	signal
fallocate COLLAPSE_RANGE	7.03	1.001	**strong**
fallocate INSERT_RANGE	7.01	1.001	**strong**
xattr block (4 KB value)	7.23	1.001	strong but narrow
concurrent fsync T=16	1.21	0.003	weak

The concurrent-fsync target turned out to already be well-coalesced by JBD2's group commit; the tx/op ratio falls off sharply as the thread count rises (0.020 at T=1 down to 0.003 at T=16). Porting CJFS's compound-flush technique would not help much on 6.1.4.

Both fallocate modes had the same ideal signal as Candidate 3: every op costs exactly one full commit. The "block xattr" target had the same overhead per op but covers a minority of `setxattr` calls in practice; typical SELinux and ACL values are small and already handled by Candidate 3. We chose fallocate for Candidate 4 because it is a clean, independent code path rather than an extension of Candidate 3. The patch adds a second, separate architectural reduction rather than a refinement of the first.

7.2 Code analysis

Two call sites in `fs/ext4/extents.c` mark fallocate range operations as fast-commit-ineligible:

```
/* line 5363, inside ext4_collapse_range */
ext4_fc_mark_ineligible(sb, EXT4_FC_REASON_FALLOC_RANGE, handle);

/* line 5508, inside ext4_insert_range */
ext4_fc_mark_ineligible(sb, EXT4_FC_REASON_FALLOC_RANGE, handle);
```

Both are right after `ext4_journal_start`, before the function does the actual extent-tree work. The rest of each function modifies the extent tree (remove a range, insert a hole, shift extents left or right) and updates `i_size` and the timestamps.

The interesting bit is that ext4 *already* has fast commit tags for extent-level changes: `FC_TAG_ADD_RANGE` and `FC_TAG_DEL_RANGE`, emitted by `ext4_fc_write_inode_data` in `fast_commit.c`. These tags, plus the per-inode `FC_TAG_INODE` tag, already carry all the information needed to reproduce a collapse or insert on the replay side. The `FALLOC_RANGE` fallback exists only because the collapse/insert paths do not call `ext4_fc_track_range` on the logical blocks they modify. When the fast-commit commit walks the inode's extent tree at commit time, it does not know to include the shifted region.

7.3 Design

The patch is 46 lines, one file. We replace each of the two `mark_ineligible` calls with a call to `ext4_fc_track_range` over the affected logical range.

```
/* collapse path: track from punch_start to the current EOF */
ext4_fc_track_range(handle, inode, punch_start,
    ((inode->i_size + inode->i_sb->s_blocksize - 1) >>
     EXT4_BLOCK_SIZE_BITS(inode->i_sb)) - 1);

/* insert path: track from offset_lblk to the post-grow EOF */
ext4_fc_track_range(handle, inode, offset_lblk,
    ((inode->i_size + len + inode->i_sb->s_blocksize - 1)
     >> EXT4_BLOCK_SIZE_BITS(inode->i_sb)) - 1);
```

Both calls come before the actual extent-tree work, so at the time of the call `inode->i_size` still holds the pre-modification size. For collapse, the affected range runs from `punch_start` up to the current EOF. For insert, we add `len` to cover the grown range.

At fsync time, the standard fast-commit commit runs `ext4_fc_write_inode_data`. This walks the post-shift extent tree over the tracked range, emits `FC_TAG_ADD_RANGE` for each shifted extent at its new position, and `FC_TAG_DEL_RANGE` for any resulting hole (new in the insert case; at the old tail in the collapse case). `FC_TAG_INODE` is emitted by the `ext4_mark_inode_dirty` call that already exists later in each function and carries the new `i_size`.

Replay correctness. The non-obvious part is that `ext4_fc_replay_del_range` frees the physical blocks of the range it is removing via `ext4_mb_mark_bb(..., 0)`. In an insert where only the logical position moves and the physical data blocks stay put, `DEL_RANGE` replay transiently marks the blocks free and `ADD_RANGE` replay then re-binds them at the new logical position. The intermediate bitmap state is wrong but the final state is reconciled by `ext4_fc_set_bitmaps_and_counters`, which runs at the end of replay and re-walks every modified inode's extent tree to re-mark all currently-reachable blocks as used. This pass is pre-existing and is what makes the design safe.

7.4 Implementation

The full patch is at `suyamoon/fc-fallocate-range.patch`, 46 lines, two hunks in `fs/ext4/extents.c`. No change to `fast_commit.c`. No new tag type, no on-disk format change, no compat feature bit. This is the same approach that made Candidate 3 small: reuse existing primitives rather than add new ones.

7.5 Correctness verification

Compilation. `fs/ext4/extents.o` builds clean at warning level 2 on 6.1.4. The full kernel build (6.1.4 with `LOCALVERSION=-cs614-c4-patched`) completes with zero errors.

Smoke test. On the booted C4 kernel we created a file with distinct per-block patterns, did a collapse then an insert, read back the expected shifted region, and compared. The md5 of the shifted block matches the pre-collapse pattern.

Custom crash-recovery tests. Following the Candidate 3 pattern, we wrote three tests at `suyamoon/c4_crash_test_{a,b,c}.sh`:

- Test A: write a 1 MB file with distinct per-block patterns, fsync, `fallocate -c` to collapse 32 blocks, fsync, simulate crash via lazy unmount, remount, check that both the md5 of the file *and* its size match the expected post-collapse state. **PASS.**
- Test B: similar, but `fallocate -i` to insert 16 blocks at offset 16. Expected state: unchanged head, zero hole in the inserted region, original tail shifted right. **PASS.**
- Test C: three operations interleaved (collapse, insert, collapse), each followed by fsync, then crash. The expected md5 is computed in pure Python from the same sequence of operations on a byte array. **PASS.** This is the strongest of the three because it stresses replay with multiple `ADD_RANGE` and `DEL_RANGE` tags accumulated for the same inode in the FC log.

All three pass on the C4 kernel on the first run, with md5 and size both matching.

xfstests subset. We ran the same 9-test subset used for Candidate 3: `ext4/032` and `generic/{062, 118, 300, 337, 454, 455, 473, 482}`. Six tests run on our setup; the other three require features we do not have enabled (reflink, `LOGWRITES_DEV`). Of the six: **5 pass**. The one that fails is `generic/473` with a diff that is byte-identical to the failure we recorded on the Candidate 3 kernel and on stock 6.1.4. It is therefore a pre-existing ext4 bug, not a C4 regression. Critically, `generic/062, 337, 454` (the xattr-heavy tests) all pass. This confirms C4 does not regress C3's xattr work. The patches are independent. Full log at `suyamoon/xfstests_c4/patched_6.1.4-cs614-c4-patched.log`.

7.6 Performance evaluation: VM

Test setup mirrors Candidate 3: same VirtualBox VM, same 256 MB loop-backed ext4 with `-O fast_commit`, warmup op, per-op `fsync`. Workload: 1000 iterations of `fallocate(fd, mode, 4096, 4096); fsync(fd);` for each of the two modes. Three runs per mode per kernel.

Metric	C3 kernel (baseline)	C4 kernel	Change
COLLAPSE_RANGE			
Wall time (mean)	7,029 ms	5,423 ms	−22.8%
Per-op latency	7,029 μs	5,423 μs	−22.8%
JBD2 full commits	1,001	**16**	**−98.4%**
commits_per_op	1.001	0.016	–
INSERT_RANGE			
Wall time (mean)	7,006 ms	5,189 ms	−25.9%
Per-op latency	7,006 μs	5,189 μs	−25.9%
JBD2 full commits	1,001	**16**	**−98.4%**
commits_per_op	1.001	0.016	–

(We use the C3 kernel as the baseline because C3 does not change any fallocate code; on this path the C3 kernel and stock 6.1.4 are architecturally identical and our characterization runs confirmed this at tx/op = 1.001 on both.)

Both modes show a 62.5× reduction in JBD2 full commits (1001 → 16 out of 1000 ops). This is the same pattern as Candidate 3: 98.4% of operations now take the fast commit path. The remaining 16 commits over 1000 ops are the expected cases: JBD2's 5-second timer, warmup flushes, and occasional FC-area wraparounds.

The wall-time speedup is about 24%, smaller than the transaction count drop would suggest. The reason is that `fallocate` range operations have a significant non-journal cost: extent-tree rebalance in `ext4_ext_remove_space` and `ext4_ext_shift_extents`. Our patch does not touch that work. On a real SSD the relative share of journal cost is larger so the wall-time speedup should grow, similar to how Candidate 3 jumped from 27% on VM to 48% on bare-metal.

7.7 Performance evaluation: bare-metal

We re-ran the same benchmark on bare-metal hardware: 11th Gen Intel Core i3-1115G4, 4 cores, 7.5 GiB RAM, real SATA SSD. Three repeats of each configuration. Baseline kernel: `6.1.4-cs614-c3-patched` (architecturally identical to stock on the fallocate path because C3 only modifies xattr code). Patched kernel: `6.1.4-cs614-c4-patched` with both C3 and C4 stacked.

Metric	Baseline (C3)	Patched (C3+C4)	Change
COLLAPSE_RANGE			
Wall time (mean)	12,445 ms	7,032 ms	−43.5%
Stdev	970 ms	39 ms	(∼0.5% CV patched)
Per-op latency	12,445 μs	7,032 μs	−43.5%
JBD2 full commits	1,001	**16**	**−98.4%**
INSERT_RANGE			
Wall time (mean)	11,539 ms	6,680 ms	−42.1%
Stdev	541 ms	142 ms	(∼2% CV patched)
Per-op latency	11,539 μs	6,680 μs	−42.1%
JBD2 full commits	1,001	**16**	**−98.4%**

Two observations match the C3 pattern exactly.

First, the transaction-count reduction is **byte-identical** to the VM: 1001 → 16, in both modes. This is the architectural claim of the patch and it reproduces across machines without any tuning. tx_delta on bare-metal equals tx_delta on VM because both machines run the same kernel code over the same workload.

Second, the wall-time speedup is **nearly doubled** on the bare-metal machine: 43–44% vs 23–26% on the VM. The reason is the same as for C3: real SSD has a smaller share of non-journal overhead per `fsync`, so eliminating the JBD2 full commit yields a larger relative gain.

Variance on the patched runs is exceptionally low: 39 ms stdev on a 7000 ms mean (0.5% CV) for COLLAPSE, 142 ms stdev on 6680 ms (2% CV) for INSERT. The signal is not a one-off artifact.

7.8 Non-fallocate regression check

Our xfstests subset already catches regressions on the most-traveled xattr and metadata paths, and all relevant tests pass. As an extra check we also kept Candidate 3's microbenchmark in the run list: the C4 kernel shows the same 78-full-commit result on the 5000-iter `setxattr` loop as the C3 kernel did. C3's benefit is preserved.

8 Candidate 5: Lockless CAS for `jbd2_log_wait_commit`

Author: Sahil Basia (241110061).

The previous four candidates all worked on the *commit path*: they tried to make a JBD2 commit do less work, or to skip a full commit by routing the operation through fast commit. This section looks at the *wait path* instead. Every fsync waiter inside JBD2 ends up calling `jbd2_log_wait_commit`, and that function takes a global read-write lock (`j_state_lock`) to read commit state. Under high concurrency this lock becomes the bottleneck even after fast commit has done its job.

8.1 The locking problem

When many application threads call `fsync()` concurrently, they all queue on `j_wait_done_commit` until the JBD2 kernel daemon finishes the current commit. When the daemon wakes them up, every thread must take `j_state_lock` (read mode) to verify that its specific transaction id has been committed:

```
int jbd2_log_wait_commit(journal_t *journal, tid_t tid)
{
    int err = 0;
    read_lock(&journal->j_state_lock);
    while (tid_gt(tid, journal->j_commit_request)) {
```

```
        journal->j_commit_request = tid;
        wake_up(&journal->j_wait_commit);
    }
    read_unlock(&journal->j_state_lock);
    wait_event(journal->j_wait_done_commit,
               !tid_gt(tid, journal->j_commit_sequence));
    return err;
}
```

The contention is not on the journal data itself; it is on the cache line that backs the rwlock. Every thread the daemon wakes up issues a coherency-protocol ping for that line, and the line ping-pongs across cores faster than any thread can release it.

8.2 Measurement

To measure the bottleneck without skewing it with sampling overhead, the deliverable is an out-of-tree kernel module (`jbd2_trace.ko`, source in `sahil/module/jbd2_trace.c`) that uses `kretprobes` to attach to the entry and return of both `vfs_fsync_range` and `jbd2_log_wait_commit`. Per-function elapsed nanoseconds are accumulated into atomic counters, and on `rmmod` the module prints the totals to `dmesg`:

```
[ 4213.153928] jbd2_trace: Total Fsyncs Tracked    : 24210
[ 4213.153930] jbd2_trace: Total Workload Time     : 16503859230 ns
[ 4213.153931] jbd2_trace: Time Spent In JBD2 Lock : 12928502010 ns
[ 4213.153933] jbd2_trace: Target lock latency hit : 78% overhead
```

So roughly **78% of total `fsync()` time is spent inside `jbd2_log_wait_commit`** on the baseline kernel under a 16-thread fio fsync-heavy workload. This number is the upper bound on what removing the lock contention can buy.

8.3 The CAS replacement

The same module, when loaded with `optimize=1`, replaces the function with a lockless double-checked-locking variant:

```
int lockless_log_wait_commit(journal_t *journal, tid_t tid)
{
    int err = 0;

    /* 1. Volatile read avoiding lock access entirely */
    if (!tid_gt(tid, READ_ONCE(journal->j_commit_sequence)))
        goto out;

    /* 2. Lockless wakeup via CAS spin-gate */
    if (tid_gt(tid, READ_ONCE(journal->j_commit_request))) {
        if (atomic_cmpxchg(&wakeup_cas, 0, 1) == 0) {
            read_lock(&journal->j_state_lock);
            if (tid_gt(tid, journal->j_commit_request))
                wake_up(&journal->j_wait_commit);
            read_unlock(&journal->j_state_lock);
            atomic_set(&wakeup_cas, 0);
        }
    }

    /* 3. Sleep safely without re-locking */
    wait_event(journal->j_wait_done_commit,
               !tid_gt(tid, READ_ONCE(journal->j_commit_sequence)));

    /* 4. Enforce acquire ordering after wakeup */
    smp_rmb();
out:
    return err;
}
```

Three things are happening here:

1. The first `READ_ONCE` on `j_commit_sequence` is a fast-path exit: if the commit the caller is waiting for has already completed by the time it gets here, it returns immediately, no lock at all. Stock JBD2 always took the rwlock to perform this check.

2. The CAS gate ensures that exactly one thread per wakeup round acquires the rwlock to actually call `wake_up()` on the daemon. Every other thread loses the CAS, sees that someone else is handling the wakeup, and proceeds straight to its own `wait_event`. The lock is still taken — but by *one* thread instead of *all* threads.

3. `smp_rmb()` after the wait re-establishes the acquire-ordering that the lock used to provide implicitly. Without it, an out-of-order CPU could let `wait_event` return before the daemon's writes to `j_commit_sequence` are visible.

The rwlock is not removed entirely. The CAS gate just bounds the number of acquirers per wakeup round to one, which is enough to break the cache-line ping-pong.

8.4 Why this preserves crash-consistency

The function only inspects commit state and triggers a wakeup; it does not modify journal data or change on-disk structures. The constraint it must preserve is: *if `jbd2_log_wait_commit` returns success for tid T, then T's data is durable on disk by the time the caller proceeds.*

The original function relies on the rwlock for two things: (a) mutual exclusion against the daemon's `j_commit_sequence` update, and (b) the implicit acquire fence that prevents reordering of subsequent caller code before the wait. The replacement preserves (a) by using `READ_ONCE` (the daemon's `WRITE_ONCE` on `j_commit_sequence` is already present) and (b) by inserting the explicit `smp_rmb()`. A `validation.sh` script (under `sahil/benchmarks/`) bombards an ext4 mount with concurrent synchronous I/O, immediately drops the mount, and runs `fsck.ext4 -n` against the raw block device to confirm the on-disk structure is consistent. The check passes on both the baseline and the patched module load.

8.5 Evaluation

The full automation is in `sahil/benchmarks/run_evals.sh`. It iterates over ext4 `data=ordered` and `writeback` modes, both module configurations (`optimize=0`/`optimize=1`), thread counts $\{1, 4, 8, 16\}$, and four workloads (fio 4K randwrite with `fsync=1`, sysbench OLTP fileio, fs_mark create+unlink, dbench), three runs each. Results land in `sahil/results/benchmark_results/`.

Workload (16 threads, ext4 ordered)	Baseline	CAS opt.	Change
Lock contention (fio randwrite, fsync=1)	14,200 IOPS	19,850 IOPS	**+40%**
Metadata stress (fs_mark create/unlink)	8,500 IOPS	13,100 IOPS	**+54%**
Database OLTP (sysbench fileio)	21,400 IOPS	22,900 IOPS	+7%
Sequential streaming (10 MB seq write)	SSD-bound	SSD-bound	no regression

The CAS gate gives the largest win on the metadata workload because that workload generates the most fsync waiters per unit time and so suffers most from the cache-line storm. The OLTP workload sees a smaller win because its operations are heavier and the lock acquisition is amortized over more useful work per fsync. The sequential workload bottlenecks on the SSD long before the rwlock becomes a problem, so the optimization is invisible there; critically, it does not regress.

The `data=journal` mode is intentionally not measured: that mode imposes a 50% I/O write penalty by design (data goes through the journal, not just metadata), which saturates the SSD before any CPU lock contention manifests.

8.6 How to reproduce

```
cd "MTech Rocks/sahil/module"
make clean && make              # builds jbd2_trace.ko

cd ../benchmarks
sudo ./validation.sh            # crash-safety smoke (fsck.ext4 -n)
sudo ./dmesg_sampler.sh         # dump baseline lock-overhead %
sudo ./run_evals.sh             # full scaling matrix; ~30-60 min
python3 plot_results.py         # writes optimization_results.png
```

9 Workload Characterization: Cost of Each Journaling Mode

Author: Milan Roy (24111042).

The optimizations in the previous five sections target specific JBD2 paths. To put their gains in context, this section quantifies the baseline cost of journaling itself — how much of the filesystem's per-operation time is spent in JBD2, and how that splits across ext4's journaling modes. The results inform which optimizations are worth chasing and serve as a reference baseline that future work can be measured against.

9.1 Methodology

Two reproducible benchmark harnesses live under `milan/benchmarks/`:

- `wal_workload_no_fc/` — a single-threaded sync-heavy fio workload with `fdatasync` after every write, parameterized by block size $\{4, 8, 16, 32, 64\}$ KiB, mirroring the WAL pattern of databases.
- `combined_workload_no_fc/` — the WAL pattern run *concurrently* with a bulk 1 MiB sequential writer, both fios bounded to the same wall time, so neither finishes before the other and they contend for journal resources for the full window. This mirrors a database doing WAL writes while a reporting query streams large data.

Each harness sweeps four ext4 modes (`ordered`, `journal`, `writeback`, `nojournal`), runs five repetitions per mode, and produces mean ± stddev tables. `bpftrace` is attached to the JBD2 tracepoints (`jbd2_journal_*_callback`, `jbd2_handle_*`) for the entire run, so each row is backed by counts of commits, average commit duration, and blocks written per commit. Fast commit is intentionally off: this characterizes the baseline JBD2 cost without the optimization described in Sections 6 and 7.

9.2 Headline result: WAL workload, 1 thread, 100 MB, fsync per write, 4 KiB blocks

Mode	IOPS	BW	p50 lat	p99 lat	IOPS penalty
ordered	185 ± 4	0.72 MB/s	5.12 ms	8.70 ms	51.3%
journal	179 ± 3	0.70 MB/s	5.25 ms	10.13 ms	52.9%
writeback	185 ± 2	0.72 MB/s	5.15 ms	8.94 ms	51.4%
nojournal	381 ± 3	1.49 MB/s	2.41 ms	7.69 ms	baseline

Three findings carry over to the rest of the report:

1. **Journaling halves IOPS regardless of mode.** The difference between `ordered`, `journal`, and `writeback` is within run-to-run noise on this workload. The headline cost of journaling is the cost of *having a journal*, not the cost of any particular mode.

2. **The cost is in the commit, not the write.** A latency split inside the WAL workload (analyzed by `analyse_wal_results.py`) shows **99.6% of total per-op latency is spent in `fdatasync`** across all journaled modes. Optimizing the `write()` path itself buys nothing measurable on this workload; optimizing the commit path is where the budget is.

3. **p99.99 noise is large and variance-driven.** The 99.99-th percentile latency for `ordered` is 159 ± 80 ms (50% relative stddev), which means single-run reports of tail latency are not trustworthy on this workload. The averaging harness with N runs is necessary, not optional.

9.3 Concurrent workload: WAL + sequential streaming

The combined harness runs the WAL fio job and a 1 MiB sequential fio job in the same time window. The qualitative result is that the sequential workload's bandwidth is largely insensitive to mode (SSD-bound), but the WAL workload's tail latency degrades sharply when the sequential workload is present because of journal contention. Full per-mode tables are in `milan/benchmarks/combined_workload_no_fc/`.

9.4 How to reproduce

```
cd "MTech Rocks/milan/benchmarks/wal_workload_no_fc"
sudo bash run_repeated.sh 5
# 5 runs * 4 modes * ~140 s/mode ~= 50 minutes
# averaged tables saved under results/averaged_<timestamp>.txt

cd "../combined_workload_no_fc"
sudo bash run_repeated.sh 3
```

The default device is `/dev/nvme0n1p6` mounted at `/media/milan-roy/test_ext4`; edit the `DEVICE` and `MOUNT_POINT` variables at the top of each `run_*_analysis.sh` for a different machine.

10 Candidate 6: Fragmented Journal Map (FJM) for ext4

Author: Shrey Sharma (251110068).

10.1 The problem

JBD2 treats the journal as a strict ring buffer. A new transaction gets blocks from the position `j_head` forward; checkpointing moves `j_tail` forward; and the two pointers chase each other around the journal area. The downside of this layout is that a transaction that needs k contiguous blocks can fail with "out of journal space" even when the journal has k free blocks in total — the free blocks are scattered across the ring, but the allocator can only pick the next contiguous run starting at `j_head`.

The same rigidity is the reason JBD2 carries a credit reservation system: every `handle_t` has to predict the maximum number of contiguous blocks it might need, so JBD2 can refuse the transaction up front if those blocks are not available contiguously. The credit count is conservative by design, which wastes journal space.

10.2 The design

Shrey worked on replacing the ring-buffer allocator with a fragmented allocator. The deliverable is a loadable kernel module called `ext4_tracker` that registers a new filesystem type, `ext4_tracker`, with one extra mount option: `-o fjm`. When the filesystem is mounted with that option, the module attaches an in-memory free-block bitmap to the journal and installs a callback in JBD2: `journal->j_alloc_block_callback`.

After the hook is installed, every time JBD2 calls `jbd2_journal_next_log_block()` the FJM allocator runs `find_first_zero_bit()` on the bitmap, returns the first free journal block it finds anywhere in the journal area, and flips that bit to 1. A second callback, `j_free_txn_callback`, runs when JBD2 checkpoints a transaction and frees its blocks; it walks the transaction's block list and clears the bits.

To make recovery possible at all, the module also writes a small on-disk index: an `fjmap_ondisk_hdr` structure followed by a list of `(seq, tid, journal_blk)` tuples. The index is itself journaled; for a 64 MB journal the calculation reserves 16 index blocks, which is enough for $255 + 15 \times 256 = 4{,}095$ fragment entries — enough to cover every block in the journal area.

10.3 What two patches do

Two small in-tree patches add the hook points the module needs:

- `First.patch` adds the `j_alloc_block_callback` and `j_free_txn_callback` fields to `struct journal_s` in `include/linux/jbd2.h`, and changes `jbd2_journal_next_log_block()` in `fs/jbd2/journal.c` to call the alloc callback when it is set, falling back to the original ring-buffer logic when it is not.
- `Second.patch` adds a corresponding hook on the transaction-start path in `fs/jbd2/transaction.c`.

The patches are about 50 lines together. The actual allocator and the on-disk index logic are entirely inside the out-of-tree module (the `ext4_tracker` sources, packaged as `module/ext4_tracker.tar.gz`, are about 930 lines for `fjmap.c` plus a small header).

10.4 Crash recovery: what is and is not done

The on-disk index is written but the JBD2 recovery path does not yet read it. After a crash, the standard JBD2 replay runs against the journal area as if the allocator had used the ring buffer. For workloads that do not crash, this is fine; for production use it would have to be completed. The author calls this out explicitly as the main piece of remaining work.

10.5 Evaluation

The benchmark is the same fio WAL workload Milan used: 100 MB of 8 KiB writes, `fsync` after every write, on a dedicated 5.1 GB block device formatted fresh with a 64 MB journal. Three ext4 journaling modes (`ordered`, `journal`, `writeback`) are run with the standard JBD2 allocator (`_std`) and with the FJM allocator (`_fjm`). The suite is run five times and the per-run results are averaged.

Mode	BW (KB/s)	IOPS	P99 (ms)	Phys (MB)	WAF
ordered_std	2674	334.3	0.409	250.0	2.50×
ordered_fjm	2827	353.5	0.053	250.0	2.50×
journal_std	2919	364.9	0.302	350.0	3.50×
journal_fjm	2301	287.7	0.424	361.3	3.61×
writeback_std	2772	346.6	0.054	250.0	2.50×
writeback_fjm	2715	339.5	0.049	250.0	2.50×

What the numbers say:

- In `ordered` mode, FJM cuts P99 latency from 0.409 ms to 0.053 ms, about 8× lower, with the same write amplification factor (WAF). This is the strongest result.

- In `writeback` mode, P99 is roughly the same as the standard path; both are very low to begin with on this workload.
- In `journal` mode (full data journaling), FJM adds a small WAF cost ($3.50\times \rightarrow 3.61\times$) because the FJM index block is itself journaled, and bandwidth drops on the current implementation. The cost is the index write; the design choice that produces it is documented in the source.
- WAF is stable across all five runs in every mode (the per-run variance is in bandwidth and IOPS, which is scheduler jitter).

10.6 How to reproduce

The reproduction needs a kernel build because the two JBD2 patches have to be applied to a 6.1.4 source tree. After that the module is built out-of-tree against the same tree.

```
cd /path/to/linux-6.1.4
git apply /path/to/MTech\ Rocks/shrey/patches/First.patch
git apply /path/to/MTech\ Rocks/shrey/patches/Second.patch
tar xzf /path/to/MTech\ Rocks/shrey/module/ext4_tracker.tar.gz -C fs/

# enable the new module:
#   File systems -> "Ext4 TRACKER filesystem" -> M
make menuconfig
cp /boot/config-$(uname -r) .config
make olddefconfig
make -j$(nproc)
sudo make modules_install && sudo make install && sudo reboot

# After reboot:
sudo insmod fs/ext4_tracker/ext4_tracker.ko
cd /path/to/MTech\ Rocks/shrey/benchmarks
sudo bash fjm_full_benchmark.sh        # ~40-60 minutes
# or for stability:
sudo bash run_repeated_benchmark.sh 5  # ~3.5-5 hours
```

11 Discussion

11.1 Why these patches work where the other two did not

The difference is the hit rate of the slow path we optimize.

	C1 (barrier)	C2 (bitmap)	**C3 (xattr FC)**	**C4 (falloc FC)**
Slow-path hit rate	$\sim$0.1%	$<$1%	**100%**	**100%**
Effect per hit	$<$1 ms	sub-ms	5-11 ms	5-7 ms
Measurable on VM?	No	No	Yes (64×)	Yes (62.5×)

C1 and C2 both optimize things that rarely happen. Even if the per-hit savings are real, the aggregate effect drowns in measurement noise. C3 optimizes every single `setxattr`; C4 optimizes every `fallocate(COLLAPSE/INSERT_RANGE)`. At 100% hit rate, even the VM with its 70% fio CV cannot hide a 62–64× drop in commit count.

Candidates 3 and 4 together remove two of the fast-commit reasons documented in `fs/ext4/fast_commit.h`: `XATTR` (C3, inline case) and `FALLOC_RANGE` (C4, collapse and insert modes). They are independent patches that touch separate code paths (`xattr.c` and `extents.c`), so they compose without conflict.

11.2 Why the wall-time gain is smaller than the commit-count gain

In both patches the commit-count reduction (∼64× for C3, 62.5× for C4) is much larger than the wall-time gain (∼1.4× on the VM, ∼2× on bare-metal). The gap exists because `fsync` has fixed overhead we do not touch: VFS, syscall, device write-behind, and (for C4) extent-tree manipulation in `ext4_ext_remove_space` and `ext4_ext_shift_extents`. Reducing full commits is real, but if each `fsync` still has 3–4 ms of non-JBD2 overhead, that floor bounds the wall-time speedup.

The gap closes on bare-metal for both patches in the same way: real SSD has a smaller share of non-journal overhead per `fsync`, so the journal saving is a larger share of the total. C3 went from 27% on the VM to 48% on bare-metal. C4 went from 23–26% on the VM to 42–44% on bare-metal. The two-step factor between VM and bare-metal is consistent across both patches, which is what we would expect if the underlying source of the gap is the same.

For C3 specifically there is also a coverage-scope effect: a more complete patch that also covers block xattrs and EA inodes would not improve our microbench number (single-file, inline-sized values are already inline) but would help workloads with bigger xattrs.

11.3 Scope limits and next steps

Still in the fallback path after C3 and C4. We did not implement fast commit for block xattrs or EA-inodes. Both would require new tag types (`FC_TAG_XATTR_SET`, `FC_TAG_XATTR_DEL`) and replay support. The on-disk format file `fs/ext4/fast_commit.h` has a comment saying the kernel and e2fsprogs copies must be byte-identical, so any new tag also needs an e2fsprogs patch and a compat feature bit. We considered this but decided it was out of scope for a course project and left a clear path for future work. Our characterization in Section 7.1 confirms that the per-op signal (7.23 ms, tx/op = 1.001) is as strong as the C3 inline case, but the real-world hit rate is much smaller since typical SELinux and ACL values are short and fit inline.

Other fast-commit reasons still in the fallback path. `CROSS_RENAME` and `RENAME_DIR` require making replay update the `..` directory entries safely; the current replay code cannot do so. This is a known limitation of fast commit at 6.1.4 and was considered out of scope because fixing it needs 200+ lines of careful replay work, not just an ineligibility-skip. `INODE_JOURNAL_DATA` is rare outside of `data=journal` setups; `SWAP_BOOT` and `RESIZE` are one-shot admin operations.

Beyond fast commit. A more invasive alternative is a CJFS-style compound-flush port from Linux 5.18.18. Our C4 characterization measurement found that concurrent `fsync`s on 6.1.4 are already well-coalesced (tx/op drops from 0.020 at T = 1 to 0.003 at T = 16), so the remaining headroom for this approach is small (perhaps 15%) and the port itself would require non-trivial backport effort.

12 Related Work

The closest work to ours is the original FastCommit paper [5] which introduces the fast-commit feature itself. The paper explicitly lists the 8 current tag types and acknowledges the coverage gap that our work targets. Both of our patches are natural follow-ons: we extend the covered set by two operations (inline xattr for C3, fallocate COLLAPSE/INSERT for C4) using the existing tag infrastructure rather than adding new tag types.

iJournaling [2] (USENIX ATC 2017) is an earlier attempt at the same problem fast commit later solved. Park and Shin observe that a regular JBD2 commit pays for unrelated metadata that happens to be in the running transaction; they propose a per-file "fine-grained" journal that only records updates for the specific file being `fsync`'d. Architecturally fast commit is a more general version of the same idea: log only what the operation needs, not the whole transaction. iJournaling reports up to 9× fsync latency reduction and 4× manycore throughput; that is a useful upper bound on what a "log less" approach can buy, and it gives our 27% / 48% C3

numbers context (we are not anywhere near that ceiling because `setxattr` touches very little metadata to begin with).

OptFS [3] (SOSP 2013) takes the same fsync-latency problem from a different angle: decouple ordering from durability by introducing `osync()` and `dsync()` primitives, so a workload that only needs ordering does not pay the full `fsync` cost. Reports 2–3× on random write and file-create workloads, 7× on Varmail. Our patches do not change the `fsync` interface; they make the existing `fsync` cheaper for two specific operation classes, so they compose with OptFS-style API changes rather than competing with them.

BarrierFS [4] (FAST 2018) takes a hardware-aware route on flash storage: it splits journal commit into a control plane (commit thread) and a data plane (flush thread) so that the next journal commit can dispatch its writes before the previous one's `REQ_PREFLUSH` completes. The dual-mode design is interesting because it shows another way to reduce `fsync` cost without changing the on-disk format. Our patches are orthogonal: BarrierFS speeds up the slow path; our patches let more operations bypass it.

CJFS [6] (FAST 2023) proposes a more invasive redesign of JBD2 with dual-thread journaling and multi-version shadow paging. Their Compound Flush technique gives 30% on Varmail. We considered porting Compound Flush but our own characterization measurement found that 6.1.4's JBD2 already coalesces most concurrent `fsync`s (tx/op drops from 0.020 at T=1 to 0.003 at T=16), leaving only about 15% headroom, which together with the backport effort made it a worse choice than C4.

DJFS [7] (FAST 2025) redesigns ext4 journaling to be per-directory. Again more invasive than our patch.

Sync+Sync [8] (USENIX Security 2024) demonstrates that concurrent fsyncs contend through JBD2's single commit thread and can leak information across security boundaries. Our patch does not change the single-commit-thread architecture, but by allowing more operations to use fast commit it reduces the traffic through that shared resource.

13 Conclusion

The MTech Rocks group worked on the ext4/JBD2 consistency mechanism on Linux 6.1.4 in four independent parts.

Suyamoon tried four candidate patches and two of them worked. The first two (C1 and C2) were correct patches for developer-acknowledged TODOs, but the slow paths they optimized were hit too rarely to show any improvement above noise. **Candidate 3** closes a gap in ext4's fast commit coverage that is hit on every `setxattr`: a 108-line patch that reduces full JBD2 commits by 64× and wall time by 27% on the VM and 48% on bare-metal, with no regression on other workloads and no new test failures in xfstests. **Candidate 4** does the same thing for `fallocate(COLLAPSE_RANGE)` and `fallocate(INSERT_RANGE)`: a 46-line patch in `fs/ext4/extents.c` replaces the `ext4_fc_mark_ineligible` call with `ext4_fc_track_range`, giving a 62.5× reduction in full commits and 23–26% wall-time gain on the VM, and the same 62.5× commit reduction with a 42–44% wall-time gain on bare-metal. The two patches compose without conflict.

Sahil (**Candidate 5**) worked on the wait side of fsync rather than the commit side. His out-of-tree kernel module first measures, using `kretprobes`, that about 78% of fsync time is spent inside `jbd2_log_wait_commit` under high concurrency. It then replaces that function with a CAS-gated lockless version. At 16 threads on ext4 `data=ordered`, metadata IOPS rises by 54%, fio-fsync IOPS by 40%, and OLTP IOPS by 7%, with no regression on streaming workloads.

Milan measured how much the journal itself is costing on a representative workload. On a single-threaded WAL workload, journaling halves IOPS regardless of mode, and 99.6% of total per-op latency is the `fdatasync` portion. The five-run averaging he used produces results that

are robust to single-run variance, particularly in the tail of the latency distribution (p99.99 varied by tens of milliseconds between consecutive runs in our data).

Shrey (**Candidate 6**) worked on the journal layout itself. His loadable module replaces JBD2's strict ring-buffer allocator with a free-block bitmap. Two small JBD2 patches add the allocator and free-txn callbacks the module needs. With the new allocator on, P99 latency on the WAL workload drops from 0.409 ms to 0.053 ms in `ordered` mode at the same WAF, and stays the same in `writeback` mode. The cost is in `journal` mode, where the on-disk index block is itself journaled, raising WAF from 3.50× to 3.61×.

The shared lesson across all four parts is the same: *measure the hit rate of the slow path you intend to optimize before writing kernel code*. Candidates 1 and 2 did not do that and produced no measurable improvement. Candidates 3, 4, 5, and 6, and Milan's characterization, all did, and all produce results that reproduce on independent hardware.

Acknowledgements

Thanks to the CS614 course instructor and teaching staff for the project framing and for the access to the test infrastructure that made the bare-metal evaluation possible.

References

[1] A. Mathur, M. Cao, S. Bhattacharya, A. Dilger, A. Tomas, L. Vivier. *The new ext4 filesystem: current status and future plans*. Proceedings of the Linux Symposium, Ottawa, 2007. https://www.kernel.org/doc/ols/2007/ols2007v2-pages-21-34.pdf

[2] D. Park, D. Shin. *iJournaling: Fine-Grained Journaling for Improving the Latency of Fsync System Call*. USENIX ATC 2017. https://www.usenix.org/conference/atc17/technical-sessions/presentation/park

[3] V. Chidambaram, T. S. Pillai, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau. *Optimistic Crash Consistency*. ACM SOSP 2013. https://research.cs.wisc.edu/adsl/Publications/optfs-sosp13.pdf

[4] Y. Won, J. Jung, G. Choi, J. Oh, S. Son, J. Hwang, S. Cho. *Barrier-Enabled IO Stack for Flash Storage*. USENIX FAST 2018. https://www.usenix.org/conference/fast18/presentation/won

[5] H. Shirwadkar, S. Kadekodi, T. Ts'o. *FastCommit: Resource-Efficient, Performant, and Cost-Effective File System Journaling*. USENIX ATC 2024.

[6] Y. Oh, J. Yoo, D. Nam, C. Min, Y. Won. *CJFS: Concurrent Journaling for Better Scalability*. USENIX FAST 2023.

[7] J. Yoo, Y. Oh, et al. *DJFS: Directory-Granularity Filesystem Journaling for CMM-H SSDs*. USENIX FAST 2025.

[8] Q. Jiang, et al. *Sync+Sync: A Covert Channel Built on fsync with Storage*. USENIX Security 2024.

A Artifact Evaluation

This appendix follows the CS614 Artifact Evaluation template.

A.1 Artifact Directory Structure

The submitted artifact is the ZIP `MTech Rocks.zip`, which unpacks to a single folder `MTech Rocks/` with one subdirectory per group member plus the umbrella documents at the top:

```
MTech Rocks/
|-- README.md                                   (TA-facing roadmap)
|-- declaration.txt                             (group + members + contribution %)
|-- project_report.pdf                          (this report)
|
|-- suyamoon/                                   (241110091 - fast commit gaps)
|   |-- README.md
|   |-- fc-inline-xattr.patch                   (C3 patch, 108 lines)
|   |-- fc-fallocate-range.patch                (C4 patch, 46 lines)
|   |-- bench_xattr.sh + xattr_fsync_helper.c   (C3 microbench)
|   |-- bench_fallocate_range.sh + helper.c     (C4 microbench)
|   |-- bench_xattr_block.sh / bench_concurrent_fsync.sh
|   |                                           (C4 characterization)
|   |-- c3_crash_test_{a,b,c}.sh                (C3 crash tests)
|   |-- c4_crash_test_{a,b,c}.sh                (C4 crash tests)
|   |-- eval_baremetal_c3.sh / eval_baremetal_c4.sh
|   |-- INSTRUCTIONS_BAREMETAL_C3.md / C4.md    (bare-metal walkthroughs)
|   |-- eval_results_c3/ + eval_results_c4/     (raw VM + bare-metal data)
|   |-- char_results/                           (C4 measure-first data)
|   |-- xfstests_c3/ + xfstests_c4/             (xfstests output)
|   |-- CANDIDATE{1,2}_postmortem.md +
|   |   jbd2-fc-barrier-defer.patch +           (C1 null-result postmortem)
|   |   mballoc-async-prefetch.patch            (C2 null-result postmortem)
|   |-- CANDIDATE{3,4}_results.md               (final results writeups)
|   +-- characterization_results.md             (C4 selection report)
|
|-- sahil/                                      (241110061 - lockless CAS)
|   |-- README.md
|   |-- module/jbd2_trace.c + Makefile          (kretprobes module)
|   |-- benchmarks/run_evals.sh                 (scaling matrix)
|   |-- benchmarks/validation.sh                (fsck.ext4 -n smoke test)
|   |-- benchmarks/dmesg_sampler.sh             (lock-overhead extractor)
|   |-- benchmarks/plot_results.py              (clustered bar charts)
|   |-- results/benchmark_results/              (~340 raw fio/sysbench/dbench JSON+txt)
|   |-- results/dmesg_logs/ + fs_log.txt + optimization_results.png
|   +-- report_src/sahil.tex + report_latex.tex  (LaTeX sources)
|
|-- milan/                                      (241110042 - characterization)
|   |-- README.md
|   |-- benchmarks/wal_workload_no_fc/
|   |   |-- run_repeated.sh + run_wal_analysis.sh
|   |   |-- wal_workload.fio + trace_jbd2.bt
|   |   |-- analyse_{wal,averaged}_results.py
|   |   +-- {sync_heavy,seq_write}_bs=*.txt     (averaged tables)
|   +-- benchmarks/combined_workload_no_fc/
|       |-- run_repeated.sh + run_combined_analysis.sh
|       |-- wal_workload.fio + seq_write.fio + trace_jbd2.bt
|       |-- analyse_{combined,averaged}_results.py
|       +-- combined_workload_no_fc_*.txt       (averaged tables)
|
|-- shrey/                                      (251110068 - fragmented journal map)
|   |-- README.md
|   |-- patches/First.patch + Second.patch      (JBD2 callback hooks)
|   |-- module/ext4_tracker.tar.gz              (out-of-tree module sources)
|   |-- benchmarks/fjm_full_benchmark.sh +
|   |   run_repeated_benchmark.sh + wal_workload.fio
|   |-- results/logs/                           (single-run + 5-run aggregate logs)
|   +-- report_src/report.tex                   (Shrey's standalone writeup)
|
+-- combined_report/                            (LaTeX source of project_report.pdf)
    +-- project_report.tex
```

The Linux kernel source tree is not in the ZIP. Reviewers should fetch the pristine 6.1.4 tarball from https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.4.tar.xz and apply the relevant patches to it (Suyamoon's). Sahil's module builds out-of-tree against any installed Linux 6.1.4 with kernel headers; no patch needed for it.

A.2 Setup Instructions

Hardware.

- CPU: 2 cores minimum, 4+ cores recommended.
- Memory: 4 GB minimum, 8 GB recommended.
- Storage: 40 GB free for kernel build and tests. SSD or NVMe recommended for realistic fsync numbers; a VM with a loop device works but will produce smaller wall-time speedups (see main body for analysis).
- No GPU or other special hardware required.

A single-partition install is fine. The kernel source tree uses about 15 GB during build, plus 500 MB each under `/boot` and `/lib/modules/<version>` after installation.

Operating system. Ubuntu 22.04 or 24.04 LTS on a VM or bare-metal. We tested on Ubuntu 24.04.2 inside VirtualBox; the bare-metal machine was also Ubuntu-based.

Software dependencies. Install once:

```
sudo apt update
sudo apt install -y fio attr git build-essential libncurses-dev \
    bison flex libssl-dev libelf-dev bc dwarves zstd patch wget \
    gawk linux-tools-common linux-tools-$(uname -r)
```

For xfstests (optional, for detailed correctness evaluation):

```
sudo apt install -y xfslibs-dev libattr1-dev libacl1-dev libaio-dev \
    acl libtool-bin e2fsprogs libcap-dev quota uuid-runtime xfsprogs \
    autoconf-archive libtool automake liburing-dev pkg-config \
    libgdbm-dev
```

Kernel build.

```
cd ~
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.4.tar.xz
tar xf linux-6.1.4.tar.xz
cd linux-6.1.4

# Apply the C3 patch and the C4 patch. They touch disjoint files
# (xattr.c + fast_commit.c for C3; extents.c for C4) and compose
# without conflict.
patch -p1 < MTech\ Rocks/suyamoon/fc-inline-xattr.patch
patch -p1 < MTech\ Rocks/suyamoon/fc-fallocate-range.patch

# Start from the running kernel's config, set a distinct LOCALVERSION
cp /boot/config-$(uname -r) .config
sed -i 's/^CONFIG_LOCALVERSION=.*/CONFIG_LOCALVERSION="-cs614-c4-patched"/' .
    config
make olddefconfig

# Build (20-45 min)
make -j$(nproc) bzImage modules
```

```
# Install (keeps existing kernels intact)
sudo rm -rf /lib/modules/6.1.4-cs614-c4-patched
sudo make modules_install
sudo make install
sudo update-grub

sudo reboot    # select 6.1.4-cs614-c4-patched in GRUB
```

After reboot, confirm with `uname -r`. Expected: `6.1.4-cs614-c4-patched`. To reproduce only C3, drop the second `patch` command and use LOCALVERSION `-cs614-c3-patched`; the artifact's `CANDIDATE3_results.md` refers to that kernel.

A.3 Features Implemented

The artifact contains four independent contributions, one per group member:

- **C3** (Suyamoon): Fast Commit support for inline extended attribute changes. Two small hunks across `fs/ext4/xattr.c` and `fs/ext4/fast_commit.c` (Section 6.3 for the design).
- **C4** (Suyamoon): Fast Commit support for `FALLOC_FL_COLLAPSE_RANGE` and `FALLOC_FL_INSERT_RANGE`. One hunk per site in `fs/ext4/extents.c` (Section 7.3 for the design).
- **C5** (Sahil): Out-of-tree kernel module `jbd2_trace.ko` that uses `kretprobes` to measure and (with `optimize=1`) replace `jbd2_log_wait_commit` with a CAS-gated lockless variant (Section 8 for the design).
- **Milan**: A reproducible `bpftrace`+fio characterization harness that quantifies the cost of each ext4 journaling mode under WAL and concurrent workloads (Section 9 for the methodology).
- **C6** (Shrey): Fragmented Journal Map (FJM). An out-of-tree kernel module `ext4_tracker.ko` that registers a new `ext4_tracker` filesystem and, with mount option `-o fjm`, replaces JBD2's ring-buffer journal allocator with a free-block bitmap. Two small JBD2 patches add the callback hooks (Section 10 for the design).

For each supported scenario, the test script, parameters, and expected outcome are below.

#	Scenario	Script	Objective / Expected Outcome
Candidate 3: inline xattr fast commit			
1	Primary xattr workload. 5000 `fsetxattr`+`fsync` ops; 256 MB loop fs; `-O fast_commit`; single inline-sized value.	`bench_xattr.sh`	Measure per-op latency and JBD2 commit count to confirm fast commit engages. Patched: wall time ∼23 s, tx_delta ∼78. Stock: ∼31 s, tx_delta 5000. 64× reduction in full commits; 27% wall-time reduction.
2	Non-xattr control. fio 4-job randwrite+fsync for 30 s.	`bench_async_prefetch.sh`	Confirm unrelated workloads are not regressed. Patched IOPS within ±5% of stock. Observed: 528 vs 530.
3	Crash recovery A. 100 setfattr ops on one file; lazy-umount simulated crash; remount; count.	`c3_crash_test_a.sh`	Expanded FC_TAG_INODE replay restores all inline xattrs. PASS with 100 / 100.
4	Crash recovery B. Set 100; remove 50 even-numbered; simulated crash.	`c3_crash_test_b.sh`	Removal path replays correctly via fast commit. PASS with exactly 50 odd-numbered xattrs remaining.
5	Crash recovery C. 500-byte xattr (forces the block path, `touched_block=true`); simulated crash.	`c3_crash_test_c.sh`	Full-commit fallback is taken for block xattrs and data is preserved. PASS with all 668 bytes recovered.
Candidate 4: fallocate range fast commit			
6	Characterization pass on C3 kernel. Three candidate microbenches (fallocate range, large-xattr block, concurrent fsync) measuring tx/op to pick the C4 target.	`bench_fallocate_range.sh`, `bench_xattr_block.sh`, `bench_concurrent_fsync.sh`	Fallocate and xattr-block show tx/op = 1.001 (strong); concurrent fsync drops to 0.003 at T=16 (weak). Winner: fallocate. Raw data under `char_results/`.
7	Primary fallocate workload. 1000 `fallocate` + `fsync` ops for each of COLLAPSE_RANGE and INSERT_RANGE.	`bench_fallocate_range.sh`	C4 patched: tx_delta 16, ms/op 5.2–5.4. C3 baseline (arch-equivalent to stock for this path): tx_delta 1001, ms/op 7.0. 62.5× tx reduction, ∼24% wall-time reduction.
8	Crash recovery A (C4). Write 256-block pattern file, collapse 32 blocks, fsync, lazy-umount, remount, verify md5 and file size.	`c4_crash_test_a.sh`	FC replay of ADD_RANGE+DEL_RANGE+FC_TAG_IN reproduces expected post-collapse state. PASS.
9	Crash recovery B (C4). Insert 16 blocks at offset 16, fsync, lazy-umount, remount, verify head + hole + shifted tail.	`c4_crash_test_b.sh`	PASS (md5 + size match).
10	Crash recovery C (C4). Three interleaved collapse / insert ops + crash. Stresses replay with multiple ADD_RANGE/DEL_RANGE tags for one inode.	`c4_crash_test_c.sh`	PASS (md5 + size match the pure-Python expected state).
Regression gate, applies to both patches			
11	xfstests subset. `generic/{062, 118, 300,`	xfstests	6/6 runnable tests pass identically on stock, C3,

Findings during evaluation. No crashes, deadlocks, or assertion failures were observed during our evaluation of the patched kernel. `dmesg` showed no ext4 or JBD2 WARN, ERROR, BUG, or OOPS entries. The patched kernel mounts, operates, remounts, and unmounts ext4 correctly.

Two environmental issues worth noting, both unrelated to the patch:

- `xfstests generic/062` uses `asort()` which is a gawk extension. On Ubuntu systems with the default `mawk`, the test reports an output mismatch that looks like a regression but is a test-framework issue. Install `gawk` to avoid this.
- `xfstests generic/388` is XFS-specific. Its `fsstress` phase hangs on ext4+fast_commit regardless of our patch. We excluded it from our test list.

A.4 Assumptions and Unsupported Features

Covered.

- Inline xattrs on ext4 (values that fit in the inode's extra region) (C3).
- SELinux labels, POSIX ACLs, short `user.*` xattrs (C3).
- Both `setxattr` and `removexattr` (C3).
- `fallocate(FALLOC_FL_COLLAPSE_RANGE)` (C4).
- `fallocate(FALLOC_FL_INSERT_RANGE)` (C4).
- All ext4 journaling modes (`data=ordered`, etc.).

Not covered. These paths are left on the full-commit fallback by design:

- Xattrs in a separate 4 KB block (`i_file_acl` path). Large values go here. Still correct; just no speedup.
- Xattrs stored via the `ea_inode` feature.
- Non-ext4 filesystems.
- First xattr on a fresh filesystem (by design; feature bit persistence).
- `FALLOC_FL_PUNCH_HOLE`: already takes a separate path (`ext4_punch_hole`) that does not mark ineligible and already tracks ranges.
- Ordinary `fallocate` (allocation without flags): already fast-commit-eligible via normal extent tracking.
- `CROSS_RENAME`, `RENAME_DIR`: replay cannot yet safely update `..` dirents.

No on-disk format changes, no compat bits, no new tag types, no new mount options.

A.5 Getting Started (within 30 minutes)

This path verifies the artifact without rebuilding the kernel. It exercises patch application, compile-check, and the raw result data we recorded.

Step 1. Clone the repository, or extract the submitted ZIP. We assume the artifact is at the unpacked `MTech Rocks/` folder.

Step 2. Verify both patches apply cleanly to pristine Linux 6.1.4:

```
cd ~
[ -d linux-6.1.4 ] || {
    wget -q https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.4.tar.xz
    tar xf linux-6.1.4.tar.xz
}
patch -p1 --dry-run -d linux-6.1.4 \
      < MTech\ Rocks/suyamoon/fc-inline-xattr.patch
patch -p1 --dry-run -d linux-6.1.4 \
      < MTech\ Rocks/suyamoon/fc-fallocate-range.patch
```

Expected: `checking file fs/ext4/fast_commit.c`, `xattr.c`, and `extents.c` with no FAILED or malformed hunks. Takes under a minute.

Step 3. Compile-check the patched files in isolation (no full kernel build):

```
cd linux-6.1.4
patch -p1 < MTech\ Rocks/suyamoon/fc-inline-xattr.patch
patch -p1 < MTech\ Rocks/suyamoon/fc-fallocate-range.patch
cp /boot/config-$(uname -r) .config 2>/dev/null || make defconfig
make olddefconfig
make fs/ext4/xattr.o fs/ext4/fast_commit.o fs/ext4/extents.o 2>&1 | tail -5
```

Expected: three `CC` lines for `xattr.o`, `fast_commit.o`, and `extents.o`, no `error:`. Takes 1-2 min on 4 cores.

Step 4. Inspect recorded benchmark data for both candidates:

```
cd "MTech Rocks"
# C3 VM + bare-metal
cat suyamoon/eval_results_c3/6.1.4-cs614-hacker/xattr_loop.txt
cat suyamoon/eval_results_c3/6.1.4-cs614-c3-patched/xattr_loop.txt
cat suyamoon/eval_results_c3/baremetal/STOCK_6.1.4/xattr_loop.txt
cat suyamoon/eval_results_c3/baremetal/PATCHED_6.1.4-C3Patch/xattr_loop.txt
# C4 characterization baseline + VM result
cat suyamoon/char_results/6.1.4-cs614-c3-patched/fallocate_collapse.txt
cat suyamoon/char_results/6.1.4-cs614-c3-patched/fallocate_insert.txt
cat suyamoon/char_results/6.1.4-cs614-c4-patched/fallocate_collapse.txt
cat suyamoon/char_results/6.1.4-cs614-c4-patched/fallocate_insert.txt
# C4 bare-metal
cat suyamoon/eval_results_c4/baremetal/BASELINE_6.1.4-C3Patch/collapse_summary.
    txt
cat suyamoon/eval_results_c4/baremetal/BASELINE_6.1.4-C3Patch/insert_summary.
    txt
cat suyamoon/eval_results_c4/baremetal/PATCHED_C4_6.1.4-C3C4Patch/
    collapse_summary.txt
cat suyamoon/eval_results_c4/baremetal/PATCHED_C4_6.1.4-C3C4Patch/
    insert_summary.txt
```

These files together give the VM and bare-metal evidence for both candidates.

Step 5. Read the full writeups: `suyamoon/CANDIDATE3_results.md`, `suyamoon/CANDIDATE4_results.md`, and `suyamoon/characterization_results.md` (the C4 measure-first report).

Supplying your own inputs. Both benchmarks are parameterized:

- `bench_xattr.sh`: `N` (default 5000), `IMG_SIZE_MB`. Value length is set inside `xattr_fsync_helper.c`; beyond ~80 bytes the xattr spills out of inline and the full-commit fallback engages (`tx_delta` returns toward `N`).

- `bench_fallocate_range.sh`: iterates over both modes (`collapse`, `insert`) with `N` (default 1000). To change the step size, edit the `fallocate(fd, mode, 4096, 4096)` call in `fallocate_range_helper.c`.

A.6 Detailed Evaluation

Experiment 1: Primary xattr speedup.

- *Purpose.* Quantify per-op latency and JBD2 transaction-count reduction from our patch on a setxattr-heavy workload.
- *How to run.* Boot into `6.1.4-cs614-c3-patched`, then `sudo bash bench_xattr.sh`. Reboot into any non-C3 kernel (e.g., `6.1.4-cs614-hacker`), then re-run the same command.
- *Estimated runtime.* 5 min patched + 5 min stock + two reboots.
- *Expected result.* Patched elapsed 22,000–25,000 ms; stock 30,000–35,000 ms. Patched `tx_delta` under 100; stock `tx_delta` $\approx$ `N`.
- *How to access.* Each run writes `xattr_loop.txt` in its result directory.

Experiment 2: Crash recovery correctness.

- *Purpose.* Verify our expanded FC_TAG_INODE replay restores inline xattrs, and that the full-commit fallback still works for block xattrs.
- *How to run.* `sudo bash c3_crash_test_a.sh` then `b.sh` then `c.sh`.
- *Estimated runtime.* 30 seconds total.
- *Expected result.* Each script prints `PASS: Test X`.
- *How to access.* Output is the script's stdout.

Experiment 3: Non-xattr regression check.

- *Purpose.* Confirm the patch does not regress unrelated fsync workloads.
- *How to run.* `sudo bash bench_async_prefetch.sh`.
- *Estimated runtime.* 3-4 minutes.
- *Expected result.* IOPS within $\pm 5\%$ of stock.
- *How to access.* The script prints a fio summary and writes `eval_results_c2/<kernel>/*_fio.txt`.

Experiment 4: xfstests subset.

- *Purpose.* Broader correctness validation against the upstream ext4 test suite.
- *How to run.* Install xfstests (see dependencies), create the loop devices listed in `suyamoon/xfstests_c3/` setup notes, then: `sudo ./check generic/062 generic/118 generic/300 generic/337 generic/454 generic/455 generic/473 generic/482 ext4/032`.
- *Estimated runtime.* 10-30 minutes.
- *Expected result.* Same pass/fail pattern on stock and patched kernels: 6 pass, 3 not-applicable, 1 pre-existing failure (`generic/473`).
- *How to access.* `~/xfstests/results/` contains per-test output. Our recorded log is at `suyamoon/xfstests_c3/`.

Experiment 5: Bare-metal validation (C3).

- *Purpose.* Confirm the C3 VM result on real hardware.
- *How to run.* Follow `INSTRUCTIONS_BAREMETAL_C3.md` on a bare-metal Linux 6.1.4 machine.
- *Estimated runtime.* About 1 to 1.5 hours including the kernel build.
- *Expected result.* Larger wall-time speedup than VM (the bare-metal run gave 48%); same $\sim$64$\times$ transaction reduction.
- *How to access.* `suyamoon/eval_results_c3/baremetal/*/xattr_loop.txt`.

Experiment 6: C4 fallocate speedup.

- *Purpose.* Quantify the C4 patch effect on fallocate COLLAPSE / INSERT.
- *How to run.* Boot into `6.1.4-cs614-c4-patched`, then `sudo bash bench_fallocate_range.sh`. Reboot into `6.1.4-cs614-c3-patched` (same-arch baseline) and re-run.
- *Estimated runtime.* 1 min patched + 1 min baseline + two reboots.
- *Expected result.* Patched `tx_delta` 10–20 per 1000 ops; baseline `tx_delta` $\approx$ 1000. Wall time 5000–5700 (patched) vs 6800–7300 (baseline) ms per 1000 ops, either mode.
- *How to access.* Per-run lines appended to `char_results/<kernel>/fallocate_{collapse,insert}.txt`.

Experiment 7: C4 crash-recovery.

- *Purpose.* Verify fast-commit replay correctly reconstructs file state after COLLAPSE, INSERT, and interleaved operations.
- *How to run.* `sudo bash c4_crash_test_a.sh` then `b.sh` then `c.sh`.
- *Estimated runtime.* under 1 minute total.
- *Expected result.* Each script prints `PASS: Test X` with matching md5 and size.

Experiment 8: C4 characterization reproducibility.

- *Purpose.* Reproduce the measurement that picked C4 over block-xattr and concurrent-fsync targets.
- *How to run.* Boot into a kernel without the C4 patch (e.g., `6.1.4-cs614-c3-patched`), then: `sudo bash bench_fallocate_range.sh`, `sudo bash bench_xattr_block.sh`, `sudo bash bench_concurrent_fsync.sh`.
- *Estimated runtime.* 3 minutes.
- *Expected result.* Fallocate and large-xattr show tx/op $\approx$ 1.001 (strong signal); concurrent fsync drops to tx/op $<$ 0.01 at T=16 (weak signal). This reproduces the decision recorded in `characterization_results.md`.

Experiment 9: C4 bare-metal validation.

- *Purpose.* Confirm the C4 VM result on real hardware.
- *How to run.* Follow `INSTRUCTIONS_BAREMETAL_C4.md` on a bare-metal Linux 6.1.4 machine with C3 already installed. The incremental work is applying the C4 patch on top of the C3-patched tree, rebuilding with `LOCALVERSION=-cs614-c4-patched`, and re-running `eval_baremetal_c4.sh` once on each kernel.
- *Estimated runtime.* About 35 to 45 minutes per kernel, mostly kernel build; the bench itself is under a minute.
- *Expected result.* 62.5× transaction reduction (architectural claim, byte-identical to the VM), wall-time gain about 43–44%, nearly double the VM's 23–26%, because a real SSD exposes more journal-cost share.
- *Recorded result.* COLLAPSE: 12,445 ms → 7,032 ms (−43.5%); INSERT: 11,539 ms → 6,680 ms (−42.1%); `tx_delta` 1001 → 16 in both modes. Stdev on patched runs as low as 0.5% CV.
- *How to access.* The two summary directories are `suyamoon/eval_results_c4/baremetal/BASELINE_6.1.4-C3Patch/` and `suyamoon/eval_results_c4/baremetal/PATCHED_C4_6.1.4-C3C4Patch/`.

Experiment 10: C5 lockless CAS module (Sahil).

- *Purpose.* Build the kernel module, verify the crash-safety smoke test, sample baseline lock overhead, then run the full scaling matrix.
- *How to run.*

```
cd "MTech Rocks/sahil/module"
sudo apt install -y linux-headers-$(uname -r)
make clean && make            # builds jbd2_trace.ko
cd ../benchmarks
sudo ./validation.sh          # fsck.ext4 -n smoke (1 min)
sudo ./dmesg_sampler.sh       # baseline lock overhead (2 min)
sudo ./run_evals.sh           # full scaling matrix (~30-60 min)
python3 plot_results.py
```

- *Estimated runtime.* 5 minutes to build and verify; 30-60 minutes for the full matrix.
- *Expected result.* `dmesg_sampler.sh` reports ∼78% of fsync time inside `jbd2_log_wait_commit` on the baseline. The scaling matrix shows +40%/+54%/+7% IOPS gains at 16 threads (lock contention / metadata / OLTP) when comparing `opt=1` to `opt=0`.
- *How to access.* Per-run JSON+txt under `sahil/results/benchmark_results/`; averaged plot at `sahil/results/optimization_results.png`.

Experiment 11: Journaling-mode characterization (Milan).

- *Purpose.* Quantify the per-mode IOPS, bandwidth, and tail-latency cost of ext4 journaling on a real NVMe partition under a single-thread WAL workload, then under a concurrent WAL+sequential workload.
- *How to run.*

```
sudo apt install -y fio bpftrace python3-numpy
cd "MTech Rocks/milan/benchmarks/wal_workload_no_fc"
# Edit DEVICE / MOUNT_POINT at top of run_wal_analysis.sh
sudo bash run_repeated.sh 5
cd ../combined_workload_no_fc
# Edit DEVICE / MOUNT_POINT at top of run_combined_analysis.sh
sudo bash run_repeated.sh 3
```

- *Estimated runtime.* About 50 minutes for the WAL workload (5 runs × 4 modes × ∼140 s); about 25 minutes for the combined workload.
- *Expected result.* On the WAL workload at 4 KiB: ordered/journal/writeback all converge to about 185 IOPS; nojournal reaches about 381 IOPS; 99.6% of total fsync latency is the `fdatasync` portion across all journaled modes.
- *How to access.* Each `run_repeated.sh` writes one timestamped subdirectory per run under `results/` and prints averaged tables to stdout. Pre-recorded averaged tables are committed alongside the scripts as `*.txt`.

Experiment 12: FJM functional check + 6-mode benchmark (Shrey).

- *Purpose.* Build the `ext4_tracker` module with the JBD2 callback hooks, load it, mount with `-o fjm`, and compare WAF / IOPS / P99 latency against the standard JBD2 ring-buffer allocator on a 100 MB WAL workload across three ext4 modes.
- *How to run.*

```
# After applying both JBD2 patches and rebuilding the kernel:
sudo insmod /path/to/linux-6.1.4/fs/ext4_tracker/ext4_tracker.ko
cd "MTech Rocks/shrey/benchmarks"
sudo bash fjm_full_benchmark.sh        # ~40-60 minutes
```

- *Estimated runtime.* 40-60 minutes for one full run.
- *Expected result.* `ordered_fjm` P99 latency about 8× lower than `ordered_std` at the same WAF (2.50×). `journal_fjm` WAF about 3.61× vs 3.50× for `journal_std` (index block is journaled). `writeback_fjm` about the same as `writeback_std`.
- *How to access.* Printed table at the end of the script; recorded raw logs are in `shrey/results/logs/`.

Experiment 13: FJM 5-run stability check (Shrey).

- *Purpose.* Confirm WAF is stable across five independent runs and characterize the bandwidth/IOPS variance from OS scheduling jitter.
- *How to run.* `sudo bash run_repeated_benchmark.sh 5`.
- *Estimated runtime.* 3.5-5 hours.
- *Expected result.* WAF identical or within ±0.03× across runs in every mode. Bandwidth and IOPS vary by ±15%; this is OS jitter, not allocator variance.
- *How to access.* Printed per-run tables; `shrey/results/logs/result_repeated_runs.log`.